JiraGen-QA Client Flow Overview

What This Solution Does

JiraGen-QA helps teams turn a JIRA requirement into ready-to-review automated test cases.

In simple terms, the system reads a JIRA ticket, understands what needs to be tested, studies the existing product code, creates test cases, checks whether those tests work, and then prepares the result for human review.

This reduces manual effort, speeds up QA support, and helps teams move from requirement to test coverage much faster.

One-Page Overview

Here is the full journey in plain language:

1. A team shares a JIRA ticket that describes a feature, update, or bug.
2. The system reads that ticket and understands the expected behavior.
3. It looks at the existing application code to understand how the product currently works.
4. It creates automated test cases based on the requirement and the current codebase.
5. It runs those tests to confirm they work correctly.
6. If something fails, it improves the test and tries again within a safe retry limit.
7. Once the tests pass, a human reviews the generated output.
8. If approved, the final result is either saved locally or sent to GitHub as a pull request.

What Goes Into the System

The solution needs a few basic inputs before it starts:

- A JIRA ticket ID
- Access to the existing product codebase
- Access to the AI service used to understand requirements and draft tests
- Optional GitHub access if the team wants pull requests to be created automatically

Step-by-Step Flow

1. Start With a JIRA Ticket

The process begins when a user provides a JIRA ticket number.

That ticket acts like the business request. It tells the system what needs to be tested by sharing the title, description, and acceptance criteria.

2. Read the Requirement

The system first reads the ticket carefully and breaks the request into testable expectations.

This means it is not just copying text. It is understanding what success should look like, what should happen in the normal case, and what should happen if something goes wrong.

3. Study the Existing Product

Before creating tests, the system looks through the existing codebase.

This helps it understand how the product is built today, where the relevant logic lives, and what patterns the current team already follows.

In client language, this step is similar to giving a new QA engineer time to study the application before asking them to write test cases.

4. Create a Test Plan

Once the requirement and the product context are understood, the system creates a simple internal plan.

This plan answers questions like:

- What should be tested?
- Which scenarios matter most?
- Where should the new test file be placed?

This planning step helps make the final output more relevant and more consistent with the existing project.

5. Generate Automated Tests

After the plan is ready, the system writes automated test code.

These tests are created to match the JIRA requirement and the current product behavior. The goal is to produce practical test cases, not generic examples.

6. Validate the Tests Automatically

The generated tests are then run automatically.

This is an important quality checkpoint. It ensures the output is not only written, but also checked in a real test run.

If the tests pass, the workflow moves forward.

If the tests fail, the system captures the errors, learns from them, updates the generated test, and tries again up to a defined retry limit.

7. Human Approval Step

Even though the solution automates much of the heavy work, a human still stays in control at the final stage.

Before anything is shared further, the generated test output is shown for review and approval.

This gives teams confidence that nothing is merged or sent forward without visibility.

8. Final Delivery

After approval, the result can end in one of two ways:

- If GitHub integration is enabled, the system creates a branch and opens a pull request.
- If GitHub integration is not enabled, the generated test file is saved locally for the team to use.

What the Client Receives

At the end of a successful flow, the client or internal team receives:

- A generated automated test file
- A validation step showing that the test was checked
- Optional pull request creation for easy review and collaboration

So the output is not just "AI-generated text." It is a ready-to-review test file that has gone through requirement understanding, product study, generation, validation, and human approval.

Simple Success and Failure Scenarios

Successful Outcome

The ideal path is:

JIRA ticket received -> tests generated -> tests pass -> human approves -> saved locally or sent as a pull request

Retry Outcome

If the first generated version does not work, the system does not stop immediately.

It reviews the failure, improves the test, and retries within a controlled limit.

Stopped Outcome

The process can also stop if:

- The generated tests keep failing after all retry attempts
- A reviewer does not approve the final output

This ensures the system remains safe and review-friendly rather than pushing low-quality output forward.

Why This Flow Matters

For non-technical stakeholders, the value of JiraGen-QA is simple:

- It saves time between requirement creation and test preparation.
- It reduces repetitive QA effort.
- It uses the existing codebase as context, so the output is more relevant.
- It validates the generated result before presenting it.
- It keeps human approval in the loop before final delivery.

In short, JiraGen-QA acts like an intelligent QA assistant that helps teams move faster while still keeping control, quality, and visibility in place.